

Connection-Oriented Transport: TCP

(Transmission Control Protocol and Reliable Data Transfer)

1 Introduction

The **Transmission Control Protocol (TCP)** is the Internet's main **connection-oriented, reliable transport-layer** protocol. It provides reliable, ordered delivery of data between two applications using several key mechanisms:

- **Error detection and correction**
- **Retransmissions and acknowledgments**
- **Sequence numbers and timers**
- **Flow control and congestion control**

TCP Specifications

TCP is standardized in:

- RFC 793 — Original TCP Specification
- RFC 1122 — Host Requirements
- RFC 2018 — Selective Acknowledgment
- RFC 5681 — Congestion Control
- RFC 7323 — Timestamps and Window Scaling

2 TCP Connection Establishment

TCP is called **connection-oriented** because both endpoints must first perform a **three-way handshake** before any data transfer begins. This process sets up initial parameters, buffers, and state variables for reliable communication.

Three-Way Handshake Steps

1. **Client** sends a SYN (synchronize) segment to initiate communication.
2. **Server** responds with SYN+ACK to acknowledge and synchronize.
3. **Client** sends ACK to confirm receipt.

After these exchanges, the connection is fully established and ready for bidirectional data transfer.

Logical Connection

TCP's connection is purely **logical**:

- Exists only within the two communicating hosts.
- Intermediate routers and switches do **not** store TCP state.
- Routers forward IP datagrams, not TCP "connections."

3 TCP Communication Characteristics

3.1 Full-Duplex Communication

TCP provides **full-duplex service**, meaning data can flow in both directions simultaneously. Each host maintains:

- a **Send Buffer** (outgoing data)
- a **Receive Buffer** (incoming data)

3.2 Point-to-Point Nature

TCP connections are always **one-to-one**. Unlike UDP multicast, TCP cannot send to multiple receivers in a single operation. *“Two hosts are company; three are a crowd.”*

4 Example: Establishing a Connection in Python

A client initiates a TCP connection using the command:

```
clientSocket.connect((serverName, serverPort))
```

- **clientSocket** — Local endpoint.
- **serverName** — Remote host address.
- **serverPort** — Target application port.

After the call:

1. Client sends SYN.
2. Server responds with SYN+ACK.
3. Client replies with ACK.

5 Data Transmission and Buffering

After connection setup, both hosts maintain:

- A **Send Buffer** – stores outgoing data awaiting transmission.
- A **Receive Buffer** – stores data received but not yet read by the application.

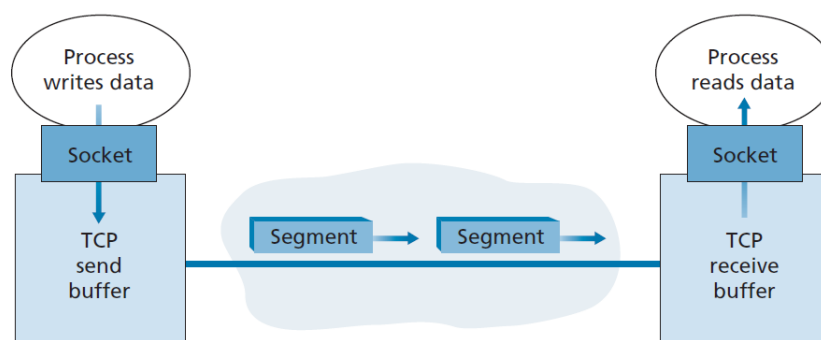


Figure 1: TCP Send and Receive Buffers

Buffer Flow

1. Application writes data into the TCP send buffer.
2. TCP segments the data and adds headers.
3. Each segment is encapsulated in an IP datagram and sent.
4. Receiver validates and stores data in its receive buffer.
5. Application reads the stream from the receive buffer.

5.1 Maximum Segment Size (MSS)

- MSS defines the largest block of application data sent in one segment.
- Determined from the link-layer **Maximum Transmission Unit (MTU)**.

Typical MTU (Ethernet) = 1500 bytes

TCP/IP Header Size $\approx$ 40 bytes

$\Rightarrow$ MSS $\approx$ 1460 bytes

Important Distinction

MSS refers only to the maximum size of the **application-layer data**, *not* the total TCP segment (which includes headers).

6 TCP Connection Structure

A TCP connection consists of:

- Buffers (send and receive)
- State variables (sequence & acknowledgment numbers, timers)
- Socket endpoints connecting the two processes

Conceptual Summary

TCP state exists only in the **end systems**. Routers, switches, and intermediate devices are **stateless** with respect to TCP connections.

7 Summary

TCP Highlights

- Connection-oriented and reliable data transfer.
- Uses three-way handshake for setup.
- Provides full-duplex and point-to-point communication.
- Employs sequence numbers, ACKs, and retransmissions.
- Maintains send and receive buffers for stream control.

In short: TCP ensures reliable, ordered delivery through buffering, acknowledgments, retransmission, and state management at both endpoints.